

METHODOLOGY

Multimodal Maritime Object Detection Using SAR and Optical Satellite Imagery

Team 22UG0005 | Panel No. 5
Guide: Mr. Nagesh Koundinya Subbanna

Overview

This methodology describes a 9-phase deep learning pipeline for all-weather maritime ship detection by fusing SAR (Synthetic Aperture Radar) and optical satellite imagery. The pipeline is sequential and reproducible, with each phase having clearly defined inputs and outputs.

Architecture Summary

SAR Backbone	ResNet50 (pretrained ImageNet)
Optical Backbone	CSPDarknet (pretrained)
Fusion Module	CrossModalAttention — channel-wise attention between SAR and optical feature maps
Neck	FPN (Feature Pyramid Network) — multi-scale feature aggregation
Detection Head	Multi-scale YOLO heads at P3 / P4 / P5 strides (8, 16, 32)
Designed Image Size	640 x 640 px (architecture standard)
Feature Channels	256 (FEAT_CH)
Anchors per Scale	3 (NUM_ANCHORS)
Classes	1 (ship)
Optimizer	AdamW with CosineAnnealingLR over 50 epochs

PHASE 0 | Data Organisation & Pairing

Objective

Establish a 1-to-1 correspondence between SAR and optical images of the same maritime scene, creating a clean, indexed dataset ready for all downstream phases.

Dataset

- Airbus Ship Detection Dataset: ~192,556 training images and ~15,000 test images (optical).
- SAR images are synthetically generated from optical images using a 6-stage optical-to-SAR simulation pipeline (radiometric profile, speckle noise, edge sharpening, texture suppression, tonal shift, contrast compression). Output: 16-bit PNG.
- All images are renamed sequentially: 0001_opt.png / 0001_sar.png, 0002_opt.png / 0002_sar.png, ...

Critical Pre-Processing Step

The ground truth CSV (train_ship_segmentations.csv) contains ~3,841 stale rows with old hex-format Imagelds from a partially interrupted rename operation. These must be removed before Phase 2.

- Filter: retain only rows where Imageld matches the sequential format.
- Result: a clean CSV with valid image-label pairs only.

Input	Output
• train_optical/ (~192K images) • test_optical/ (~15K images) • train_ship_segmentations.csv (raw)	• train_SAR/, train_OPTICAL/ (paired) • test_SAR/, test_OPTICAL/ (paired) • Cleaned CSV (stale rows removed)

PHASE 1 | Preprocessing

Objective

Standardise and clean both image modalities to a consistent input resolution. All augmentation and preprocessing is applied on-the-fly inside the DataLoader — no augmented images are written to disk.

SAR Preprocessing

- Radiometric calibration — correct for sensor gain variations.
- Speckle noise reduction — Lee filter (5x5 kernel) to suppress multiplicative noise.
- Normalisation — min-max scaling to [0, 1].
- Resize to target resolution using bilinear interpolation.

IMPORTANT: No rotation, flipping, or geometric augmentation on SAR images. YOLO label files encode bounding box coordinates tied to the original image orientation. Geometric transforms without re-computing labels cause mismatched ground truth and corrupt training.

Optical Preprocessing

- Resize to target resolution.

- Brightness and contrast jitter (+-20%) — applied randomly per batch.
- Mild random crop-and-resize within target bounds.
- No flipping or rotation (to maintain consistent pairing with SAR).

Input

- Paired raw SAR + optical images

Output

- Cleaned, normalised SAR tensor
- Augmented optical tensor
- Ready for DataLoader

PHASE 2 | Annotation & Label Generation

Objective

Convert the run-length encoded (RLE) ship masks in the CSV into YOLO-format bounding box label files. One .txt label file is generated per image, shared by both the SAR and optical streams (same scene, same bounding box).

RLE Decoding Process

- Read EncodedPixels from the cleaned CSV.
- Decode RLE: pairs of (start_pixel, run_length) fill a flat array of size height x width.
- Reshape the flat array to a 2D binary mask.
- Call cv2.boundingRect() on each connected component to get pixel-space (x, y, w, h).
- Normalise to YOLO format: $x_center = (x + w/2) / img_w$, $y_center = (y + h/2) / img_h$, $w_norm = w / img_w$, $h_norm = h / img_h$.
- Write one line per ship: class x_center y_center width height (class = 0).
- Images with NaN EncodedPixels (no ships) get an empty .txt file — background images are essential for reducing false positives.

```
# 0001.txt — three ships
0 0.452 0.671 0.112 0.083
0 0.223 0.318 0.074 0.061
0 0.789 0.512 0.095 0.078
```

Input

- Cleaned CSV with RLE masks
- Paired image index

Output

- One YOLO .txt label file per image
- Empty .txt for background images

PHASE 3 | Dataset Loading

Objective

Build a PyTorch Dataset and DataLoader that serves paired (SAR tensor, optical tensor, label tensor) triplets efficiently during training.

Subsampling Strategy

A stratified subset is drawn for feasible training:

- Positive samples: image pairs containing at least one ship.
- Negative samples: equal number of background (no-ship) pairs.

This balance prevents the model from learning to always predict no ship.

DataLoader Configuration

- pin_memory=True for faster data transfer.
- num_workers=2 on Windows (higher values cause multiprocessing issues).
- 90% train / 10% validation split using torch.utils.data.random_split.

Input • Paired image directories • YOLO label .txt files • Stratified image index	Output • (sar_tensor, optical_tensor, labels) per batch • train_loader and val_loader
--	--

PHASE 4 | Feature Extraction — Dual-Stream Backbone

Objective

Extract modality-specific deep feature maps from SAR and optical images using two independent CNN backbones, each specialised for its input modality.

SAR Backbone — ResNet50

- Input: SAR tensor [B, 1, H, W] (single-channel, normalised).
- Architecture: ResNet50 with pretrained ImageNet weights (first layer modified to accept 1-channel input).
- Output: multi-scale feature maps at stride 8 (P3), stride 16 (P4), stride 32 (P5).
- First 2 residual blocks frozen for initial epochs, then unfrozen for fine-tuning.

Optical Backbone — CSPDarknet

- Input: Optical tensor [B, 3, H, W] (3-channel RGB).
- Architecture: CSPDarknet (YOLOv5/YOLOv8 style) with pretrained weights.
- Output: multi-scale feature maps at the same three pyramid levels (P3, P4, P5).

- First 2 blocks frozen for initial epochs, then unfrozen.

Backbone Freezing Schedule

Freezing early layers for initial epochs prevents the detection head and fusion module from receiving unstable, rapidly shifting features before they have learned a meaningful representation.

Input • SAR tensor [B, 1, H, W] • Optical tensor [B, 3, H, W]	Output • SAR feature maps: Fs_P3, Fs_P4, Fs_P5 • Optical feature maps: Fo_P3, Fo_P4, Fo_P5
---	--

PHASE 5 | Multimodal Fusion — CrossModalAttention

Objective

Fuse SAR and optical feature maps at each pyramid level using a cross-modal attention mechanism, allowing the model to learn which spatial regions from one modality reinforce or suppress features from the other.

Fusion Architecture

- Concatenate SAR and optical feature maps channel-wise at each level: $F_{concat} = \text{concat}(F_s, F_o)$.
- Pass through CrossModalAttention: a lightweight transformer-style module computing attention weights between modalities.
- The fused maps at P3, P4, P5 are passed to the FPN neck.

FPN Neck

- Feature Pyramid Network aggregates multi-scale fused features.
- Top-down pathway: upsamples P5 and merges with P4, then P3.
- Output: three enhanced feature maps at each scale, all with FEAT_CH=256 channels.

Input • Fs_P3/P4/P5 (SAR features) • Fo_P3/P4/P5 (Optical features)	Output • Fused FPN feature maps: F_P3, F_P4, F_P5 (each [B, 256, H, W])
---	--

PHASE 6 | Detection Head

Objective

Apply YOLO-style multi-scale detection heads to the fused FPN feature maps. Each head independently predicts bounding boxes, objectness scores, and class probabilities at its respective scale.

Detection Head Design

- Three parallel heads, one per FPN level: P3 (small objects, stride 8), P4 (medium, stride 16), P5 (large, stride 32).
- Each head outputs a prediction tensor of shape [B, NUM_ANCHORS * (5 + NUM_CLASSES), H/stride, W/stride].
- For each anchor per grid cell: x_offset, y_offset, width, height, objectness_score, class_confidence.
- NUM_ANCHORS=3, NUM_CLASSES=1 (ship only). Final output vector per anchor = 6 values.

Anchor Configuration

- P3 (stride 8): small anchors for distant or small ships.
- P4 (stride 16): medium anchors for typical vessel sizes.
- P5 (stride 32): large anchors for supertankers and carrier vessels.

Input • Fused FPN feature maps F_P3, F_P4, F_P5	Output • Raw prediction tensors at 3 scales • Decoded bounding boxes (x, y, w, h, obj, cls) per anchor
--	---

PHASE 7 | Loss Computation & Training

Objective

Train the full model end-to-end by computing a composite detection loss and backpropagating gradients through detection heads, fusion module, and backbones.

Loss Function

- L_cls — Binary cross-entropy on class predictions (ship vs background).
- L_bbox — CloU loss on predicted vs ground truth bounding boxes. Penalises centre distance, aspect ratio, and overlap simultaneously.
- L_obj — Binary cross-entropy on objectness scores.

$$L_{total} = L_{cls} + L_{bbox} + L_{obj}$$

Training Configuration

- Optimiser: AdamW (weight decay = 0.01).
- Scheduler: CosineAnnealingLR (smoothly decays learning rate to near zero).
- Checkpoint saving: every 5 epochs. Best model saved by validation mAP.
- Validation split: 10% of training data via random_split, evaluated each epoch.

Input

- Predicted bounding boxes (Phase 6)
- Ground truth YOLO labels (Phase 2)
- Model parameters

Output

- Updated model weights per epoch
- Per-epoch train loss, val loss, val mAP logged to CSV

PHASE 8 | Evaluation

Objective

Quantitatively measure the trained model detection performance on the held-out test split using standard object detection metrics.

Metrics

- mAP@0.5 — Mean Average Precision at IoU threshold 0.5. Primary metric.
- Precision — of all predicted boxes, what fraction correctly overlaps a ground truth box.
- Recall — of all ground truth ships, what fraction was detected.
- IoU (Intersection over Union) — per-box spatial overlap score used to classify predictions as TP or FP.

Evaluation Procedure

- Load best checkpoint (highest val mAP during training).
- Run inference on the 10% validation split (no ground truth leakage from training).
- Apply NMS with IoU threshold 0.45 to remove duplicate detections.
- Compute precision-recall curve and integrate (area under curve = AP). Average over images = mAP.

Input

- Best model checkpoint
- Validation set SAR + optical pairs
- Ground truth labels

Output

- mAP@0.5, Precision, Recall per epoch
- Precision-recall curve
- Confusion matrix (ship vs background)

PHASE 9 | Inference

Objective

Deploy the trained model for real-world ship detection. Given a new SAR + optical image pair, produce annotated output images with predicted bounding boxes and confidence scores.

Inference Pipeline

- Load best checkpoint weights into the full model.
- Preprocess input: apply same SAR normalisation and optical resize as Phase 1 (no augmentation).
- Forward pass: generate raw predictions from all three detection heads.
- Post-process: decode anchor-relative offsets to absolute coordinates, apply confidence threshold (0.5), apply NMS.
- Visualise: draw predicted bounding boxes on the optical image and SAR image separately.
- Save annotated images for report and demo.

What is Not Done Here

- No ground truth is used — inference is purely prediction-only.
- No loss is computed.
- No weight updates occur.

Input • New SAR + optical image pair (unseen) • Trained model checkpoint	Output • Optical image with predicted ship bounding boxes overlaid • SAR image with same predicted boxes overlaid • Confidence score per detection
---	---

Experimental Setup & Hardware Constraints

The following section documents the actual hardware configuration used for implementation and the specific parameter choices made to ensure feasibility on local hardware. The architecture described in this methodology is designed and specified for full-scale deployment. The values below represent the training configuration used for this project submission.

Hardware Specifications

Device	Lenovo IdeaPad Slim 5 14IAH8
Processor	12th Gen Intel Core i5-12450H (8 cores, 2.00 GHz)

RAM	16.0 GB (15.7 GB usable)
GPU	Intel UHD Graphics (128 MB — integrated, no CUDA)
Storage	954 GB (715 GB used)
OS	Windows 11 Home 64-bit
Training Mode	CPU only (no GPU acceleration)

Training Parameter Adjustments

Due to the absence of a dedicated GPU, the following parameter values were used during training. These are hardware-driven adaptations and do not affect the architectural design of the system.

Parameter	Value Used	Reason
IMAGE_SIZE	320 x 320	Reduces memory by ~75% vs 640x640. Ships remain detectable at P3 (40x40 grid). Must be multiple of 32.
BATCH_SIZE	4	Prevents RAM overflow on 16GB system during CPU training.
Training Subset	10,000 pairs	Stratified: 5,000 ship + 5,000 background. Full 192K infeasible on CPU.
NUM_EPOCHS	20	Realistic on CPU (~45-90 min/epoch). Full 50-epoch run identified as future work.
NUM_WORKERS	2	Windows multiprocessing limit. Higher values cause instability.
Mixed Precision	Disabled	torch.cuda.amp has no effect on CPU-only systems. Removed to avoid overhead.
Backbone Freezing	All epochs	Kept frozen throughout training to reduce compute. Unfreeze only if time permits.
torch.set_num_threads	8	Forces PyTorch to use all 8 CPU cores. Not set by default.
Gradient Accumulation	2 steps	Simulates effective batch size of 8 while keeping memory usage of batch size 4.

Reproducibility Note for Full-Scale Deployment

To reproduce this system at full architectural scale (640x640, batch size 8, 50 epochs, full 192K dataset), the following infrastructure is recommended:

- GPU: NVIDIA RTX 3060 or higher (minimum 8 GB VRAM).
- RAM: 32 GB system RAM.
- Estimated full training time: 6-12 hours on a single RTX 3060.
- No code changes required — only the constants IMAGE_SIZE, BATCH_SIZE, NUM_EPOCHS, and training subset size need updating.
- Mixed precision (torch.cuda.amp) should be re-enabled for GPU deployment.
- Backbone unfreezing after epoch 10 should be reinstated for best accuracy.

Academic Note

The architectural design, fusion strategy, loss formulation, and evaluation methodology described in this document are independent of hardware constraints and represent the full proposed system. The experimental setup above documents the practical implementation undertaken within the constraints of student-grade hardware. This distinction is standard practice in academic deep learning research.